8. Leer hasta fin de línea (tokens variables)

"Leer hasta fin de línea" significa que lees una línea entera del teclado (o archivo) como un solo string, y luego dentro de esa línea vas extrayendo los números uno por uno.

¿Cuándo se usa?

- Cuando **no sabes cuántos números vienen por línea**
- Cuando las líneas tienen **cantidad variable de tokens**
- Para problemas donde los **datos están separados por espacios**, pero en líneas de longitud variable

Entrada:

```
1 2 3 4
5 6
7 8 9
```

Código:

```
string line;
while (getline(cin, line)) {
   stringstream flujo(line);
   int x;
   while (flujo >> x) {
      // procesar datos
   }
}
```

En programación, un **token** es una **unidad mínima con significado** dentro de una cadena de texto, separada por **delimitadores** (como espacios, comas, saltos de línea, etc.).

Ejemplos de **token:** Imagina una frase: "El perro corre rápido"
Los **tokens** (palabras) serían: "El", "perro", "corre", "rápido"

En el contexto de números: Entrada: "10 20 30 40"
Tokens (números): 10, 20, 30, 40

stringstream ss(line); ¿Qué significa "convertir la línea en un flujo"?
stringstream es una herramienta de C++ que **trata un string como si fuera un flujo de entrada** (como cin), permitiéndote extraer datos de él usando >>.
Imagina que tienes una **cadena de texto**: "10 20 30"
Y quieres sacar los números uno por uno. stringstream convierte esa cadena en un **"flujo"** donde los números flotan y puedes sacarlos con >>. Y meterlos cada uno en una variable, por ejemplo:

Ejemplo 1

```
string linea = "10 20 30";        // Cadena de texto normal
stringstream ss(linea);           // Convierte el string Linea en un "flujo"

int x;
ss >> x;  // x = 10  (extrae primer número)
ss >> x;  // x = 20  (extrae segundo número)
ss >> x;  // x = 30  (extrae tercer número)
```

Ejemplo 2

```
string linea = "42 100 3.14 Hola";
stringstream ss(linea);
int a, b;
float c;
```

string **d**;
ss >> **a** >> **b** >> **c** >> **d**; *// ¡Todo extraído ordenadamente!*

Ahora las variables son:
a =42
b =100
c =3.14
d =”hola”
Un **flujo (stream)** es una **secuencia de datos que se leen o escriben de manera continua, como si los datos “fluyeran” uno detrás de otro**. Un flujo permite procesar información poco a poco en lugar de manejar todo al mismo tiempo. Por ejemplo, **cin** es un flujo de entrada que recibe datos del teclado, **cout** es un flujo de salida que envía datos a la pantalla, y **stringstream** convierte un string en un flujo para poder leerlo igual que si fuera un archivo o la entrada estándar. Gracias a los flujos, puedes **extraer datos** usando operadores como **>>** o **funciones como getline**, leyendo números, palabras o líneas de forma ordenada y secuencial.

Extraer uno por uno manualmente

```
string linea = "10;20;30";
stringstream flujo(line);
string n1, n2, n3;

getline(flujo, n1, ';');
getline(flujo, n2, ';');
getline(flujo, n3, ';');

cout << n1 << '\n';
cout << n2 << '\n';
cout << n3 << '\n';
```

Usar for (similar a while)

```
string linea = "10;20;30";
stringstream flujo(line);
string numero;

for (int i = 0; i < 3; i++) {  // sabes que hay 3 números
   getline(flujo, numero, ';');
   cout << numero << '\n';
}
```

La analogía del agua 💧
Un flujo (stream) es como el agua que fluye por una tubería:

- cin → Es como la **llave de tu casa**: el agua (datos) viene del exterior (teclado) y tú la recoges en tu cubeta (variables).
- stringstream → Es como tener una **botella de agua cerrada**: el agua ya está dentro, pero tú decides cómo y cuándo sacarla, gota a gota o de golpe.
- **El operador** >> → Es como **abrir la llave un momento**: deja pasar el siguiente "chorro" (palabra) hasta encontrar un espacio (separador).
- getline() → Es como **llenar toda una jarra**: sigues recogiendo agua hasta encontrar un carácter específico (como un punto y coma ;).

Ejemplo:

```
string linea = "10;20;30";      // Una botella con agua etiquetada "10;20;30"
stringstream flujo(line);       // Abres la botella, el agua está lista para fluir

string numero;
while (getline(flujo, numero, ';')) {  // Vas sirviendo en vasos hasta ';'
   cout << numero << '\n';          // Te tomas cada vaso: "10", luego "20", luego "30"
}
```

Nota: Así como el **agua no vuelve hacia atrás en la tubería**, el **flujo avanza siempre hacia adelante**. **Cuando extraes un dato**, ese "agua" **ya no está disponible en el flujo**.

Pero hay otros operadores que leen flujos:

get(): Lee UN carácter.
Ejemplo: char c; cin.get(c);

peek(): Mira el siguiente carácter SIN extraerlo.
Ejemplo: char c = cin.peek();

read(): Lee bytes directamente.
Ejemplo: char buffer[10]; cin.read(buffer, 10);

readsome(): Lee algunos caracteres disponibles.
Ejemplo: cin.readsome(buffer, 5);

gcount(): Devuelve cuántos caracteres fueron leídos.
Ejemplo: cin.read(buffer, 10); cout << cin.gcount();

ignore(): Descarta caracteres del flujo.
Ejemplo cin.ignore();

Ejemplo con operador **getline** se usa para **leer una línea completa** de texto, incluyendo espacios, hasta encontrar un salto de línea (\n).
Sintaxis: **getline**(flujo, variable, delimitador);

- **Flujo**: De dónde leer (**cin**, **un archivo**, **stringstream**, etc.)
- **Variable**: Dónde guardar la línea (string)
- **delimitador (Opcional)**: Carácter que indica el final (por defecto es '\n')

Ejemplo 1: Con cin

```
string linea;
getline(cin, linea);    // Lee toda la línea hasta que presiones Enter
```

Ejemplo 2: cin con delimitador personalizado:

```
string linea;
getline(cin, linea, ',');  // Lee hasta encontrar una coma
```

Ejemplo 3: Con fichero

```
ifstream archivo("datos.txt");
string linea;
getline(archivo, linea); // Lee una línea del archivo
```

Comparativa de flujos comunes:

Flujo	Cabecera	Dirección	Destino/Origen
cin	<iostream>	Entrada	Teclado
cout	<iostream>	Salida	Pantalla
cerr	<iostream>	Salida (**error**)	**Pantalla** (sin buffer)
clog	<iostream>	Salida (**log**)	**Pantalla** (con buffer)

ifstream	<fstream>	Entrada	Archivo
ofstream	<fstream>	Salida	Archivo
fstream	<fstream>	Entrada/Salida	Archivo
stringstream	<sstream>	Entrada/Salida	String
istringstream	<sstream>	Entrada	String
ostringstream	<sstream>	Salida	String

```
#include <sstream>

getline("Hola mundo", linea); // ✗ ERROR: "Hola mundo" no es un flujo


// Leer desde string, como convertir una cadena en un flujo
string datos = "123 456 789";
istringstream flujo(datos);
int x;
flujo >> x;  // x = 123

// Escribir desde un flujo a un string
ostringstream flujo;
flujo << "El resultado es " << 42;
string resultado = flujo.str();  // "El resultado es 42"
```

Ejemplo 1: (getline con separador espacio por defecto)

```
string linea;
while (getline(cin, linea)) {          // Lee línea completa hasta EOF
   stringstream flujo(line);              // Convierte la línea en un flujo
   int x;                                 // Variable temporal
   while (flujo >> x) {                   //  >>  Extrae cada número de la línea
      // procesar números (ej: guardar, sumar, etc.)
   }
}
```

Ejemplo 2: (getline con símbolo coma). Puedes usar otros separadores: , - * +, etc

```
string s = "10,20,30";
stringstream flujo(s);
string palabra;
while (getline(flujo, palabra, ',')) {
// Convertir el token en flujo
   stringstream convertir(palabra);              // Convierte la línea en un flujo
```

```
    int x;
    // Extraer número
     while (convertir >> x) {
          cout << "Numero = "<< x<< '\n';
      }
}

int main() {
    //este es un texto que esta separado por puntos
    string texto = "Juan corre rápido. Ana estudia física. El perro duerme tranquilo";

    // flujo_oraciones:
    // Convierte el string en un flujo (como si fuera cin o un fichero)
    stringstream flujo_oraciones(texto);

    string oracion;

    //dividir el texto en oraciones:
    //getline lee hasta encontrar un punto '.'
    //Cada vuelta del ciclo obtiene UNA oración
    while (getline(flujo_oraciones, oracion, '.')) {
        cout << "oracion = "<< oracion<< '\n';

        // Convierte la oración en otro flujo
        // para leer palabra por palabra
        stringstream convertir(oracion);
        string palabra;

        // Recorre el flujo palabra por palabra
        // El operador >> separa automáticamente por espacios
        while (convertir >> palabra) {
              // También podría hacerse así:
             // while (getline(convertir, palabra, ' ')) {

            cout << "palabra = "<< palabra<< '\n';
        }

    }
    return 0;
}
```

https://dmoj.uclv.edu.cu/problem/2023occcop10c

Entrada

Línea 1: se darán 4 enteros n (columnas), m (filas), x e y.

Línea 2..2+n: n líneas de m longitud con un signo de punto (.) para representar el espacio vacío o un signo de número (#) para representar la paca de tabaco en la pirámide.

```
    int n, m, x, y;
    string linea;

    getline(cin, linea); // Línea 1
    stringstream ss(linea);
    ss >> n >> m >> x >> y;

   int i = 0;
    while (i < n) {
        getline(cin, linea); // Línea 2..2+n

   // Saltar líneas vacías
   if (linea.empty()) continue;

// Validar que la línea tenga exactamente m caracteres
        while (linea.length() != m) {
            getline(cin, linea);
        }
        i++;
    }

    return 0;
}
```

🧠 9) Múltiples casos con línea en blanco / separadores

El programa debe procesar varios conjuntos de datos (casos) hasta que ya no haya más entrada.

- Una línea vacía indica que terminó un grupo de datos y empieza otro.
- Los datos dentro de cada grupo pueden venir separados por: **espacios, comas, puntos y coma, tabs, etc.**

¿Cuándo se usa?

Por la naturaleza de la entrada:

- Problemas con varios ejemplos de entrada
- Cuando no te dicen cuántos casos hay
- Procesar archivos con muchos registros

Por la estructura de los datos:

- Problemas de bloques de texto
- Cuando los **casos están** separados visualmente
- Procesar **párrafos**

Por el formato de los datos:

- Archivos CSV (comas)
- Configuraciones con formato específico
- **Datos de** diferentes fuentes (cada fuente usa su propio separador)

es decir que si t=5, aunque escriba 1000 lineas en blanco,
estoy obligado a escribir al menos 5 lineas con textos

Entrada típica:

```
10,20,30      ← Caso 1 (separado por comas)
40 50 60      ← sigue caso 1 (separado por espacios)

              ← Línea en blanco (separador entre casos)

100;200;300   ← Caso 2 (separado por punto y coma)
```

Código:

```
int t;
cin >> t;
cin.ignore();          // limpia el salto de línea

while (t--) {
   string line;
   getline(cin, line);  // puede ser línea vacía
   if (line.empty()) {  // saltar separadores
      t++;              // no consumas caso
      continue;
   }
   // procesar 'line' o leer más líneas según el caso
}
```

👉 Muy común en problemas tipo UVA viejos.

Múltiples casos, con líneas en blanco
Se ignorar líneas vacías

Línea en blanco como separador de casos

Datos:

```
10 20 30

40 50 60

70 80 90
```

```
string linea;
while (getline(cin, linea)) { // Lee hasta EOF
    if (linea.empty()) continue;  // Salta líneas vacías

    // Procesar la línea no vacía
    cout << "Línea: " << linea << endl;
}
```

No se ignorar líneas vacías, se usa para marcar que terminó un bloque/caso. Aquí vacío TIENE significado

Datos:

```
dato1 dato2
dato3 dato4

dato5 dato6

dato7 dato8
```

```
string linea;
int caso = 1;
while (getline(cin, linea)) {
    if (linea.empty()) {
        cout << "Fin del caso " << caso++ << endl;
        continue;
    }
    // Procesar la línea no vacía
    cout << "Línea: " << linea << endl;
}
```

3. Patrón completo para leer bloques separados por línea en blanco

- detecta líneas vacías
- separa casos
- guarda TODOS los datos
- crea bloques de datos

```
datos
1 2 3
4 5

10 20

7 8 9
```

```
int main() {
    string linea;
    vector<vector<int>> bloques;    // Guarda TODOS los bloques/casos
    vector<int> bloque_actual;      // Guarda el bloque/caso actual

    while (getline(cin, linea)) {    // Leer líneas hasta EOF
        if (linea.empty()) {        // Detecta líneas vacías(indica fin de bloque)
            if (!bloque_actual.empty()) { //reviso si variable bloque tiene datos(para indicar fin de caso)
```

```
            bloques.push_back(bloque_actual); // Guarda el bloque completo
            bloque_actual.clear();// Limpia para comenzar un nuevo bloque
        }
    }
    else {
        stringstream ss(linea); // Convierte la línea en flujo
        int x;
        while (ss >> x) {  //separa cada token
            bloque_actual.push_back(x); // Guarda los datos en el bloque actual
        }
    }
}


// (por si el archivo no termina con línea vacía). Guarda el último bloque
if (!bloque_actual.empty()) {
    bloques.push_back(bloque_actual);
}

// Impresión de los bloques guardados
for (int i = 0; i < bloques.size(); i++) {

    cout << "Bloque " << i + 1 << ": ";
    for (int x : bloques[i]) {
        cout << x << " ";
    }
    cout << '\n';
}
return 0;
}
```

4. Diferentes separadores según contexto

```
string linea = "10,20,30"; //separadores por comas, CSV
stringstream ss(linea);
string campo;
while (getline(ss, campo, ',')) {
    cout << campo << " ";
}

string linea2 = "10\t20\t30"; //separadores de tabulación TSV
stringstream ss2(linea2);
while (getline(ss2, campo, '\t')) {
    cout << campo << " ";
}


string linea3 = "10|20|30"; // separadores Pipe
```

```
stringstream ss3(linea3);
while (getline(ss3, campo, '|')) {
    cout << campo << " ";
}
```

5. Saltar líneas vacías

✓ lee múltiples casos hasta EOF
✓ salta líneas vacías

```
XOX
.O.
XXO

OOX
XXO
OX.
int main() {

 string parrafo =
R"(Este es un párrafo.
Tiene varias líneas.
Y conserva los saltos de línea.)";

    string oracion;
    stringstream flujo(parrafo);


    while (true) {
        // Leer primera línea, si está vacía saltar

        //ignorar las líneas en blanco, y si se tropieza con el fin de archivo sale de todo
        do {
            if (!getline(flujo, oracion)) return 0;   //Fin de archivo
        } while (oracion.empty()); //

        cout << "Linea leida: " << oracion << '\n';
    }
}
```

Procesa bloques fijos (3x3), ignorando líneas vacias

Funciona para:

- **Tic-Tac-Toe (3x3 fijo)**
- **Cualquier matriz 3x3**

```
#include <iostream>
#include <string>
#include <sstream>

using namespace std;

int main() {
   string parrafo =
R"(ABC

DEF

GHI

XO
OX
XO

123
456
789)";

   stringstream flujo(parrafo);
   char tablero[3][3];
   string linea;

   while (true) {
      // Leer las 3 filas del tablero, ignorando líneas en blanco
      for (int i = 0; i < 3; i++) {
         do {
            if (!getline(flujo, linea))
               return 0;  // Fin de archivo
         } while (linea.empty());  // Ignorar líneas en blanco

         for (int j = 0; j < 3; j++) {
            tablero[i][j] = linea[j];
         }
      }

      // Mostrar tablero
      cout << "\nTablero:\n";
      for (int i = 0; i < 3; i++) {
         for (int j = 0; j < 3; j++) {
            cout << tablero[i][j] << ' ';
         }
         cout << '\n';
      }

   }

   return 0;
}
```

Procesa bloques fijos (MxN), ignorando líneas vacias

✓ útil para Tic Tac Toe, Sudoku, mapas, laberintos (dimensiones variables)

```
#include <iostream>
#include <string>
#include <sstream>
#include <vector>

using namespace std;

int main() {
   string entrada =
R"(3 3
ABC
DEF
GHI

4 4
ABCD
EFGH
IJKL
MNOP

9 9
530070000
600195000
098000060
800006003
400803001
700020006
060000280
000419005
000080079)";

   stringstream flujo(entrada);
   int filas, columnas;
   string linea;

   while (flujo >> filas >> columnas) {   //1ra linea de cada bloque: 3 3

      // Leer el salto de línea después de filas y columnas
      getline(flujo, linea);

      //Cada iteración del ciclo es un ámbito NUEVO
      //Ámbito (scope) y tiempo de vida
      //Cada vez que entras al ciclo, se crea una variable NUEVA y la anterior deja de existir.
      //Es un vector de vectores = MATRIZ DINAMICA
      vector<vector<char>> matriz(filas, vector<char>(columnas));

      // Leer la matriz
      for (int i = 0; i < filas; i++) {
         do {
            if (!getline(flujo, linea))
               return 0;
         } while (linea.empty());
```

```
        for (int j = 0; j < columnas && j < linea.length(); j++) {
            matriz[i][j] = linea[j];
        }
    }

    // Mostrar
    cout << "\nMatriz " << filas << "x" << columnas << ":\n";
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            cout << matriz[i][j];
            if (j < columnas-1) cout << ' ';
        }
        cout << '\n';
    }
  }

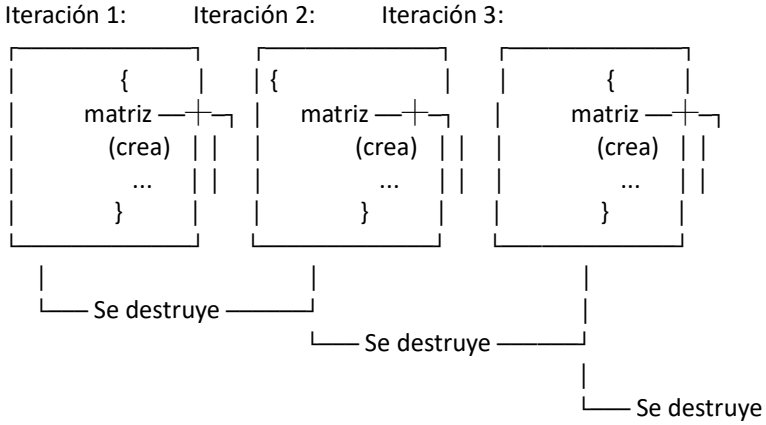
  return 0;
}
```

Cada iteración del ciclo es un **ámbito NUEVO**, Ámbito (scope) y tiempo de vida
Cada vez que entras al ciclo, se crea una variable NUEVA y la anterior deja de existir.
Por ello es que se puede crear la MATRIZ DINAMICA dentro del ciclo

```
while (true) {
    // ← Aquí COMIENZA un ámbito nuevo
    vector<vector<char>> matriz(...);
   // ...
} // ← Aquí TERMINA el ámbito, TODO se destruye

// La siguiente iteración comienza con un ámbito FRESCO y NUEVO
```

Visualización gráfica:



<table>
<tr><td colspan="2">Es un patrón muy común para saltar líneas vacías hasta encontrar una línea con datos o llegar al final del archivo.</td></tr>
<tr><td>do {
Revisar si hay carta(); // Intentar leer

if (No hay más cartas()) {
Terminar día(); // return 0
}

} while (La carta está en blanco()); // Si está en blanco, seguir revisando</td><td>string linea;
do {
if (!getline(cin, linea)) {
return 0; // No hay más datos
}
} while (linea.empty()); // Saltar líneas vacías

// Aquí 'linea' tiene la primera línea NO vacía
cout << "Primera línea con datos: " << linea << endl;</td></tr>
<tr><td>• Revisas el buzón (getline)
• Si ya no hay más cartas (EOF) → terminas
• Si la carta está en blanco → sigues revisando
• Si la carta tiene contenido → te quedas con ella</td><td>Iteración 1: Lee línea vacía → está vacía → repetir
Iteración 2: Lee línea vacía → está vacía → repetir
Iteración 3: Lee "Hola mundo" → NO está vacía → SALIR
// El programa continúa con 'linea' conteniendo "Hola mundo"</td></tr>
<tr><td></td><td>Archivo de entrada:
← Línea 1: vacía (se salta)
← Línea 2: vacía (se salta)
Hola mundo ← Línea 3: con datos (se queda)
Adiós ← Línea 4: ya no entra al bucle</td></tr>
</table>

10) método de entrada por bloques o casos

¿Qué significa?

Es un formato de entrada donde **cada caso de prueba está compuesto por bloques de datos de tamaño conocido**, típicamente:

1. **Primero** lees un número **T = cantidad de casos**
2. **Para cada caso**, lees:
   - N = tamaño del primer bloque
   - N **líneas/datos** del primer bloque
   - **M = tamaño del segundo** bloque
   - M **líneas/datos** del **segundo** bloque

Es como **leer "paquetes" de datos** donde primero te dicen cuántos vienen, luego los datos, luego otro paquete, etc. Muy común en problemas de programación competitiva.

Entrada: (cada caso tiene 2 bloques)

```
3              ← T = 3 casos
2              ← Caso 1: N = 2
Línea 1 del caso 1
Línea 2 del caso 1
1              ← Caso 1: M = 1
Dato 1 del caso 1
---
3              ← Caso 2: N = 3
Línea 1 del caso 2
Línea 2 del caso 2
Línea 3 del caso 2
2              ← Caso 2: M = 2
Dato 1 del caso 2
Dato 2 del caso 2
---
1              ← Caso 3: N = 1
Línea 1 del caso 3
0              ← Caso 3: M = 0 (no hay datos)
```

Problema: *Leer T equipos, cada equipo tiene N jugadores y M suplentes*

Código:

```
int t; cin >> t;
while (t--) {

  int n; cin >> n;
   vector<string> jugadores(n);
   for (int i = 0; i < n; i++) cin >> jugadores[i];

   int m; cin >> m;
   vector<int> suplentes(m);
   for (int i = 0; i < m; i++) cin >> suplentes[i];
}
```

Aquí falla gente que mezcla mal lecturas.

Ejemplo: Leer estudiantes y sus calificaciones
Entrada:

- *La **primera línea** contiene un **entero T** (1 ≤ T ≤ 100), que representa el número de cursos a procesar.*
- *Para cada curso:*
  - ***Una línea** con un entero **N** (1 ≤ N ≤ 50), que indica la **cantidad** de **estudiantes**.*
  - ***Luego N líneas**, cada una con el **nombre** de un estudiante.*
  - *Una línea con un entero M (0 ≤ M ≤ 30), que indica la cantidad de materias.*
  - *Luego M líneas, cada una con el nombre de una materia.*

2 3 Juan Maria Carlos 2 Matematica Fisica 2 Ana Luis 1 Historia	*Caso 1:* *Estudiantes (3):* *- Juan* *- Maria* *- Carlos* *Materias (2):* *- Matematica* *- Fisica* *Caso 2:* *Estudiantes (2):* *- Ana* *- Luis* *Materias (1):* *- Historia*

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
   // Entrada simulada
   string entrada =
R"(2
3
Juan
Maria
Carlos
2
Matematica
Fisica
2
Ana
Luis
1
```

```
Historia)";

   // En programa real sería: cin >> ...
   // Aquí simulamos con stringstream
   #include <sstream>
   stringstream cin(entrada);

   int T; cin >> T;
   int caso = 1;

   while (T--) {
      cout << "Caso " << caso++ << ":\n";

      // Bloque 1: Estudiantes
      int N; cin >> N;
      vector<string> estudiantes(N);
      cout << "  Estudiantes (" << N << "):\n";
      for (int i = 0; i < N; i++) {
         cin >> estudiantes[i];
         cout << "    - " << estudiantes[i] << '\n';
      }

      // Bloque 2: Materias
      int M; cin >> M;
      vector<string> materias(M);
      cout << "  Materias (" << M << "):\n";
      for (int i = 0; i < M; i++) {
         cin >> materias[i];
         cout << "    - " << materias[i] << '\n';
      }
      cout << '\n';
   }

   return 0;
}
```

11) Lectura hasta un “END” (centinela string)

Lees datos hasta encontrar una palabra especial (centinela) que indica "aquí termina la entrada". Sigue leyendo hasta que te diga 'ya terminé' con la palabra mágica END". Muy útil cuando el usuario decide cuántos datos ingresar.

Código:

```
string s;
while (cin >> s && s != "END") {
   // procesar
}
```

Muy usado en strings.

Centinelas comunes "END", "FIN", "0", "-1", "***"

```
// 1. Centinela diferente
while (cin >> s && s != "FIN")
while (cin >> s && s != "***")
while (cin >> s && s != "0")  // 0 como centinela

// 2. Centinela case-insensitive
while (cin >> s && s != "END" && s != "end")

// 3. Múltiples centinelas
while (cin >> s && s != "END" && s != "EXIT" && s != "QUIT")

// 4. Para líneas completas
while (getline(cin, linea) && linea != "END")
```

¿Cuándo Usarlos?

Cantidad desconocida de datos

```
// No sabes cuántos nombres vendrán, solo que terminan con "END"

string nombre;
while (cin >> nombre && nombre != "END") {
    cout << "Registrando: " << nombre << '\n';
}
```

Procesar líneas hasta "END"

```
string linea;
while (getline(cin, linea) && linea != "END") {
   cout << "Línea: " << linea << '\n';
}
```

Múltiples casos terminados por "END"

```
int caso = 1;
string palabra;
while (cin >> palabra && palabra != "END") {
   cout << "Caso " << caso++ << ": " << palabra << '\n';
}
```

😵 12) Lectura con doble condición (EOF + validación)

SIMILAR al 😵 11), pero con doble condición

¿Qué significa?

Es una lectura que **termina cuando ocurre UNA de dos condiciones**:

1. **EOF** (End of File) - No hay más datos
2. **Centinela** - Se encuentra un valor especial (ej: n == 0)

Sigue procesando mientras puedas leer datos Y **los datos sean válidos** (no sean el centinela)

Este patrón se usa cuando:

- Los **datos** vienen en **pares o grupos**
- Hay un valor **centinela** que indica "**fin**"
- Necesitas **validar rangos**
- Quieres **ignorar** datos **inválidos**
- Los casos de prueba pueden variar en cantidad

Entrada:

n m
...
Pero algunos casos inválidos deben ignorarse.

Código:

```
int n, m;
while (cin >> n >> m && n != 0) {
   // procesar
}
```

//Sumar pares de números hasta que aparezca 0 0

```
int a, b;
while (cin >> a >> b && (a != 0 || b != 0)) {
   cout << "Suma: " << a + b << '\n';
}
```

Ignorar casos inválidos automáticamente

```
// Solo procesar cuando n > 0
int n, m;
while (cin >> n >> m && n > 0 && m > 0) {
   cout << "Área: " << n * m << '\n';
}
// Si n o m son negativos o cero, se ignoran y termina
```

Validar rango de valores

```
// Procesar edades válidas (1-120)

int edad;
while (cin >> edad && edad >= 1 && edad <= 120) {
   cout << "Edad válida: " << edad << '\n';
}
// Si edad es 0, negativa o >120, termina
```

```
// 1. Validar rango
while (cin >> n && n >= 1 && n <= 100)

// 2. Múltiples centinelas
while (cin >> n && n != -1 && n != 0)

// 3. Centro de mesa (0 0)
while (cin >> x >> y && (x != 0 || y != 0))

// 4. Valores positivos
while (cin >> n >> m && n > 0 && m > 0)

// 5. Hasta encontrar un número específico
while (cin >> palabra && palabra != "FIN" && palabra != "END")
```

🧠 13) Leer todo el input (casos raros)

Es leer **TODO el archivo de entrada** sin ninguna estructura predefinida, guardando **cada línea** en un vector para **procesarlo manualmente después**.

Algo así como: Trágate todo el archivo línea por línea, luego yo me encargo de entender qué hacer con esto

¿Cuándo se usa? (CASOS RAROS)

- Formato desconocido o variable
- Validación o preprocesamiento complejo
- Ignorar líneas inválidas o comentarios
- Reordenamiento necesario

📥 Entrada: archivo completo sin estructura clara

✅ Código:

```
vector<string> all;
string s;
while (getline(cin, s)) {
   all.push_back(s);
}
```

👉 Luego procesas tú mismo

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
   vector<string> lineas;
   string linea;

   // Leer todo, y guardarlo todo en un vector
   while (getline(cin, linea)) {
      lineas.push_back(linea);
   }

   // Procesar ignorando comentarios y líneas vacías
  //proceso donde manualmente veo que trae línea por línea
   cout << "Líneas válidas:\n";
   for (const string& l : lineas) {
      if (!l.empty() && l[0] != '#') {
         cout << "  " << l << '\n';
      }
   }

   return 0;
}
```

```
// Buscar líneas que contengan "ERROR" o "WARNING"
    cout << "Problemas encontrados:\n";
    for (size_t i = 0; i < lineas.size(); i++) {
        if (lineas[i].find("ERROR") != string::npos) {
            cout << "Línea " << i+1 << ": " << lineas[i] << '\n';
        } else if (lineas[i].find("WARNING") != string::npos) {
            cout << "  Advertencia en línea " << i+1 << '\n';
        }
    }
```

```
// Determinar qué tipo de entrada es
    if (linea.size() == 1) {
        // Una sola línea - podría ser un comando
        cout << "Comando simple: " << linea[0] << '\n';
    }
    else if (linea.size() <= 10 && linea[0].find(' ') != string::npos) {
        // Pocas líneas con espacios - podría ser matriz pequeña
        cout << "Posible matriz de " << linea.size() << "x? \n";
        for (const string& fila : linea) {
            cout << "  " << fila << '\n';
        }
    }
    else {
        // Muchas líneas - podría ser código o texto
        cout << "Texto largo con " << linea.size() << " líneas\n";
    }
```